Open Source Formal Verification

SBY

Yann Thoma

Reconfigurable and Embedded Digital Systems Institute
Haute Ecole d'Ingénierie et de Gestion du Canton de Vaud



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License

November 2026

1. Introduction
2. gtkwave and outputs

Three modes that SBY offers

- **BMC** (bounded model checking)
- **Prove** (unbounded proof)
- **Cover** (reachability)

SBY mode: BMC

- Checks assertions from the initial states up to a chosen depth k
- A counterexample within the bound demonstrates a property violation
- No counterexample through k is only a bounded PASS, not a proof for all executions
- Increasing the depth allows for more comprehensive checking, but at the cost of increased computational resources

SBY mode: Prove

- Attempts to prove assertions for all reachable executions, not just a fixed depth
- PASS establishes the properties for the encoded design and assumptions
- A counterexample demonstrates a violation; timeout or UNKNOWN means no conclusion
- Proof engines may not converge, and a difficult proof can take substantial resources

SBY mode: Cover

- Searches for an execution that reaches each cover goal
- A reached goal produces a witness trace and demonstrates reachability
- It does not check that assertions are correct
- A goal not reached within the search depth may need a larger bound; this alone does not prove it unreachable

SBY

- SBY is run through a script defining various information
  - `[tasks]` declares task names and optional tags
  - `[options]` sets each task's mode and options, such as the BMC depth
  - `[engines]` selects the engine(s) used for each task
  - `[script]` contains the commands to import and prepare the design
  - `[files]` lists source and support files copied into each task's work directory

The documentation can be found here:
`https://symbiyosys.readthedocs.io/en/latest/reference.html`

Complete example: next1

run.sby (save in `cours/code/next1/scripts`)




```
[tasks]
bmc
prove

[options]
bmc: mode bmc
bmc: depth 20
prove: mode prove

[engines]
bmc: abc bmc3
prove: abc pdr

[script]
ghdl --std=08 sequencer.vhd psl_sequence.vhd -e psl_sequence
prep -top psl_sequence

[files]
../src_vhdl/sequencer.vhd
../src_vhdl/psl_sequence.vhd
```

Example

Tasks

```
[tasks]
cover
bmc
prove
```

- We can define as many tasks as we want

Example

Options

```
[options]
cover: mode cover
bmc: mode bmc
bmc: depth 20
prove: mode prove
```

- The `bmc: depth 20` option limits the `bmc` task's bounded exploration to 20 model steps
- A BMC PASS means no failure was found within that bound; it does not prove the property for all executions
- We define, for each task, its mode (cover, bmc or prove)
  - Yes, here the task names are identical to the modes
  - Yes, we can have more than one task per mode

Example

- Maybe we are aware of an assertion that fails
  - During the development process
  - Still interesting to push and use CI
  - But the CI will not be happy with the failure
  - So... We can say that a specific task is supposed to fail

Options - bmc is expected to fail

```
[options]
bmc: expect fail
```

Example

Engines

```
[engines]
cover: smtbmc z3
bmc: abc bmc3
prove: abc pdr
```

- For each task, we define the engine

Example

Script

```
[script]
ghdl --std=08 -fpsl sequencer.vhd psl_sequence.vhd -e psl_sequence
prep -top psl_sequence
```

- The GHDL-Yosys front end analyzes the VHDL and imports the elaborated design into Yosys
- `-std=08` for VHDL-2008 compatibility
- `-fpsl` to interpret PSL in VHDL comments (prefix: `--psl`)
  - Not used in this training
- `-e <entity>` identifies the top entity
- `prep -top <entity>` runs Yosys preparation on the imported design, selecting the top entity for formal processing

Example

Script - generics

```
[script]
ghdl --std=08 -gDATASIZE=8 sequencer.vhd psl_sequence.vhd -e psl_sequence
prep -top psl_sequence
```

- `ghdl` accepts generics in the form `-g<GENERICNAME>=value`

Example

Files

```
[files]
../src_vhdl/sequencer.vhd
../src_vhdl/psl_sequence.vhd
```

- All the listed files are copied by SBY prior to running the script commands

Tags

- The tasks can define tags
  - Space-separated items following the task name
- In each section, tags can be used to add some information

Example: Tasks

```
[tasks]
bmc0 ERRNO0 bmc
bmc1 ERRNO1 bmc
prove ERRNO0
```

2 tags are ERRNO0 and bmc

2 tags are ERRNO1 and bmc

Tags

Example: options and engines

```
[options]
bmc: depth 30
bmc: mode bmc
bmc1: expect fail
~prove: depth 30
prove: mode prove

[engines]
bmc: abc bmc3
prove: abc pdr
```

Applied for all bmc

Applied only to one bmc

Another option applied to everything except prove

Applied for all bmc

Tags

Example: script

```
[script]
# Applies to bmc0 and prove
ERRNO0: ghdl --std=08 -gERRNO=0 sequencer.vhd psl_sequence.vhd -e psl_sequence
# Applies to bmc1
ERRNO1: ghdl --std=08 -gERRNO=1 sequencer.vhd psl_sequence.vhd -e psl_sequence
prep -top psl_sequence
```

Some SBY hints

- Possibility to add a prefix to the output directories:

```
sby --prefix <myPrefix> <sbyFile>
```

- Possibility to run it in a temporary folder:

```
sby -t <sbyFile>
```

- Possibility to use symbolic links instead of file copy:

```
sby --link <sbyFile>
```

Interpreting SBY results

- **PASS**: the task completed successfully
  - BMC: no assertion failure was found within the configured depth
  - Prove: the selected engine established the assertions
  - Cover: the requested cover statement(s) were reached
- **FAIL**: the task's expected result was not obtained; inspect the report and trace
  - For assertion checking, a counterexample shows a behavior violating an assertion
  - For cover, failure means the requested cover point was not reached within the search bound
- **ERROR**: setup, tool, or runtime error prevented a valid result
- **UNKNOWN**: the engine could not determine the result (for example, an inconclusive or timed-out run)

gtkwave

- SBY writes a trace when an assertion failure is found or a cover point is reached (subject to engine support)
- Use the VCD path reported in the task log; it is task- and engine-specific, commonly under `<workdir>/<task>/engine_0/`
- Open the generated file, often named `trace.vcd` or `trace0.vcd`, with GTKWave: `gtkwave <path_reported_by_sby>.vcd`
- A counterexample trace demonstrates a violating behavior; a cover trace demonstrates reachability, not correctness

Testbench

- Some engines also generate a Verilog testbench alongside a trace, often named `trace_tb.v`
- Use the task- and engine-specific path reported by SBY (for example, under `<workdir>/<task>/engine_0/`)
- The testbench can be used to replay the recorded trace in a compatible simulator; it is not a substitute for interpreting the formal result